LAB ASSIGNMENT

NLP Foundations: Handwriting OCR Evaluation

Deepening your understanding after Lab 6

Course	Artificial Intelligence
Practical	Lab 6 of 8 - Handwriting OCR Evaluation
Student details	Name: Muhammad Asyraaf bin Mustafa Student ID: 24005305

Purpose

This assignment is to be completed after the Lab 6 practical. You do not need to repeat the entire experiment. Instead, use what you learned to explain the concepts, interpret the results, reason about OCR errors and suggest improvements. Answer in your own words.

Instructions

1. Complete the assignment individually.
2. Use your Lab 6 practical outputs and the results discussed during the lab as evidence.
3. Show all calculation steps. Round numerical answers to 2–3 decimal places where appropriate.
4. You may refer to your lab results; you do not need to rerun the complete OCR experiment.
5. Include screenshots or small tables only where requested. Ensure that all evidence is readable.

Part A - Understand the concepts

Q1. NLP pipeline

Describe the five stages of an NLP pipeline: acquire, normalise, represent, model and evaluate. For each stage, give an example of what happens when a handwritten medicine-name image is processed.

Acquire: Gathering the data (e.g., scanning a paper prescription to create an image file).

Normalise: Preprocessing the input to standardise it (e.g., converting the image to grayscale and binarizing it).

Represent: Extracting features from the data (e.g., identifying contours or stroke patterns from the handwritten text).

Model: Applying an algorithm to make predictions (e.g., passing the features through Tesseract to generate a text string).

Evaluate: Measuring system performance (e.g., comparing the predicted text to a human-transcribed reference using Character Error Rate).

Q2. How OCR works

Explain the difference between segmentation and character recognition. Give one example of a failure at each stage. Why is it useful to identify which stage caused an error?

Segmentation isolates individual text lines, words, or characters from the image background, whereas character recognition assigns specific digital text characters to those isolated segments. For instance, segmentation failure is the system groups two written letters together as a single block and recognition failure such as system correctly isolates an 'a' but predicts it as an 'o'. Identifying the stage is critical for troubleshooting because segmentation errors require better image preprocessing while recognition errors is important to model fine-tuning.

Q3. Text normalisation

The reference is “Tab Azimax 40mg” and the OCR output is “tab azimax 40 mg”. Apply the Lab 6 normalisation rule (lowercase and keep only letters and digits). Show both normalised strings. Why must the same rule be applied to both?

Normalised Reference: tabazimax40mg.

Normalised Prediction: tabazimax40mg.

The same rule must be applied to both strings so that evaluation metrics measure core recognition accuracy rather than penalizing the model for inconsistent styling, spacing, or capitalization.

Q4. Normalisation trade-off

The Lab 6 normaliser removes spaces. Describe one situation where removing spaces could hide an important difference. Suggest how you would change the normalisation rule for that situation.

Removing spaces could hide important difference such as confusing "10 mg" with "100mg" if a character is misread or combining two distinct words in a way that creates a different medicine name. A better normalisation rule would retain single spaces while removing only multiple spaces or punctuation.

Part B — Work with OCR measurements

Q5. Levenshtein distance

Calculate the Levenshtein distance between “breeky” (reference) and “beeky” (prediction). Identify the edit operation(s). Then calculate the normalised edit distance by dividing the distance by the length of the longer string.

The Levenshtein distance between "breeky" and "beeky" is 1. The edit operation is a deletion of the letter 'r'. The normalised edit distance is 1 / 6 which equals 0.167.

Q6. Character Error Rate (CER)

An OCR system produces 4 substitutions, 2 deletions and 1 insertion. The reference contains 35 characters. Calculate CER using CER = (S + D + I) / N. Explain what the result means. Can CER be greater than 1? Explain briefly.

CER = (4 + 2 + 1) / 35 = 0.20. This means 20% of the characters in the reference string require an edit to match the OCR output. Yes, CER can exceed 100% if the OCR model hallucinates and produces significantly more insertions than the total number of characters in the original reference text.

Q7. Compare the metrics

Explain exact-match accuracy, mean normalised edit distance, CER and blank predictions. For each metric, state whether a higher or lower value is generally better. Why is exact-match accuracy alone insufficient?

Exact-match accuracy: The proportion of predictions that perfectly match the reference. Higher is better.

Mean normalised edit distance: The average edit distance across all predictions and scaled by string length. Lower is better.

CER: The rate of character-level errors. Lower is better.

Blank predictions: The count of images where OCR returned no text. Lower is better. Exact-match accuracy is insufficient because it treats a prediction with 1 error identically to a prediction with 10 errors.

Part C — Interpret the Lab 6 experiment

Use the results from Lab 6. Both pipelines were evaluated on the same 89 labelled images, with the same normalisation rule and fixed Tesseract configuration.

Q8. Read the results

Describe at least two improvements shown by the preprocessed results. Use values from the table as evidence.

The number of blank predictions decreased significantly from 42 in the baseline to 15 in the preprocessed pipeline. Moreover, the CER improved from 0.883117 (baseline) to 0.806494 (preprocessed). Additionally, this metric decreased from 0.868732 to 0.785072 showing the predictions required less edits to match the reference text.

Q9. Explain the result pattern

CER and mean normalised edit distance improved, and blank predictions decreased. However, exact-match accuracy increased only from 0.000 to 0.011. Explain how preprocessing can help recover text without making the complete medicine name correct.

Preprocessing steps improve contrast and isolate text. It allows the OCR engine to detect text blocks it previously missed and accurately recognize more individual characters. However, handwriting is unique and can be hard to read. Recognizing every single character to achieve an exact match remains highly difficult even though preprocessing helps the engine recover parts of the word correctly. This results in many partial matches but very few perfect matches.

Q10. Controlled experiment

Why did the lab keep the Tesseract settings (--oem 1 --psm 7), the 89 images and the normalisation rule unchanged? What would be difficult to conclude if several settings changed at once?

All of that being unchanged to isolate independent variable which is image preprocessing. By keeping everything else constant, any change in performance can be directly attributed to the preprocessing steps. If multiple variables were changed simultaneously, it would introduce puzzling variables, making it impossible to conclude which specific change caused the metric improvements.

Q11. Diagnose the failure

Identify the likely category for each case and explain:

(a) Tesseract raises an error while processing an image;

This is an Execution error. It typically occurs when the input file is corrupted, the image format is unsupported, or there is a software compatibility issue preventing the engine from executing.

(b) OCR returns an empty string;

This is a Detection error. The engine fails to distinguish the handwritten text from the background. This often happens due to poor contrast, extreme noise, or faint ink.

(c) OCR returns “nidu” when the reference is “nims”.

This is a Character Recognition error. The segmentation stage successfully located the text block but the model misclassified individual characters. This happens because cursive strokes or overlapping letters cause the model to confuse similar-looking shapes.

Part D — Apply your understanding

Q12. Suggest improvements

Suggest TWO improvements you would investigate next to improve handwritten OCR. For each, explain which error it targets and how you would test whether it helps. Do not claim an improvement works without evidence.

The first improvement I would investigate is applying adaptive thresholding which targets segmentation errors and blank predictions caused by uneven lighting. For this improvement, I would test its effectiveness by measuring the reduction in blank predictions and Character Error Rate (CER) on a specific subset of poorly lit images before and after implementation.

The second improvement I would investigate is domain-specific model fine-tuning which targets character recognition and substitution errors. I would test this by fine-tuning Tesseract on a medical handwriting dataset and comparing its exact-match accuracy and CER against the baseline model.

Q13. Medical safety scenario

A clinic wants to use OCR to read medicine names and dosages from handwritten prescriptions. Explain at least THREE risks of using OCR output without human checking. Recommend a safer workflow.

Relying solely on OCR output poses severe risks such as dispensing the wrong medication, administering a lethal incorrect and missing critical patient allergy warnings. A safer workflow treats OCR as a "human-in-the-loop" assistant. The OCR digitizes the prescription into a draft field but a licensed pharmacist must visually verify the prediction against the original image and manually approve it before dispensing.

Q14. Reflection

What is the most important thing you learned from Lab 6? Identify one concept you initially found difficult and explain it using an example from the practical.

The most critical takeaway is that single-metric evaluations are misleading. We need to evaluate granular metrics like CER is necessary to track model progress. A historically difficult concept is understanding how strings are mathematically compared but Levenshtein distance explains this by breaking down differences into insertions, deletions, and substitutions to quantify exact model errors.